

AWS Solutions Architect Associate (SAA-C03)

Complete Learning Guide — Chapters 1 & 2

Roadmap Reference: Phase 1 — Cloud Foundations

Units Covered: U00 · AWS Fundamentals & Global Infrastructure | U01 · IAM Essentials

Difficulty: Beginner

Goal: Build a solid foundation before touching any AWS service

Table of Contents

- [Chapter 1 — AWS Fundamentals & Global Infrastructure](#)
 - [1.1 Cloud Computing Models](#)
 - [1.2 AWS Global Infrastructure](#)
 - [1.3 Edge Locations and Points of Presence](#)
 - [1.4 Global vs. Regional vs. AZ-Scoped Services](#)
 - [1.5 AWS Shared Responsibility Model](#)
 - [1.6 AWS Service Health Dashboard & Personal Health Dashboard](#)
 - [1.7 AWS Well-Architected Framework — Six Pillars Overview](#)
 - [Chapter 1 Labs](#)
 - [Chapter 1 Exam & Interview Questions](#)
 - [Chapter 2 — IAM Essentials](#)
 - [2.1 IAM Core Concepts](#)
 - [2.2 IAM Policies](#)
 - [2.3 Amazon Resource Names \(ARNs\)](#)
 - [2.4 Multi-Factor Authentication \(MFA\)](#)
 - [2.5 IAM Best Practices](#)
 - [2.6 IAM Password Policy, Account Alias & Credential Report](#)
 - [2.7 IAM Policy Simulator](#)
 - [Chapter 2 Labs](#)
 - [Chapter 2 Exam & Interview Questions](#)
-

Chapter 1 — AWS Fundamentals & Global Infrastructure

Overview

Before you can design solutions on AWS, you need to understand the foundation everything is built on: what cloud computing actually is, how AWS has spread its infrastructure around the world, who is responsible for what in the security model, and the framework AWS uses to measure architectural quality.

Think of this chapter as learning the map and the rules of the road before you start driving.

1.1 Cloud Computing Models

What Is Cloud Computing?

Cloud computing is the **on-demand delivery of IT resources** — servers, storage, databases, networking, software — over the internet with **pay-as-you-go pricing**. Instead of buying and maintaining physical hardware in your own data centre, you rent what you need from a cloud provider like AWS.

The five essential characteristics (NIST definition) you should know:

1. **On-demand self-service** — provision resources yourself, no human interaction needed.
 2. **Broad network access** — accessible over the internet from any device.
 3. **Resource pooling** — AWS serves multiple customers from the same physical infrastructure (multi-tenancy).
 4. **Rapid elasticity** — scale up or down in minutes, not months.
 5. **Measured service** — you pay only for what you consume.
-

The Three Cloud Service Models

IaaS — Infrastructure as a Service

You get raw computing infrastructure: virtual machines, storage, and networking. You manage the OS, middleware, runtime, data, and applications. AWS manages the physical hardware.

Real-world analogy: Renting an empty warehouse. You bring your own furniture, layout, and staff. The building owner only handles the building itself.

AWS Examples: EC2 (virtual machines), EBS (block storage), VPC (networking)

When to use IaaS:

- You need full control over the OS and configuration
- You're migrating existing on-premises applications
- You have a DevOps team that can manage servers

PaaS — Platform as a Service

AWS manages the infrastructure AND the runtime platform (OS, middleware, runtime). You only manage your application code and data.

Real-world analogy: Renting a fully furnished apartment. You just bring your personal belongings (the app). The landlord handles the building, plumbing, furniture maintenance.

AWS Examples: Elastic Beanstalk, RDS (managed database), AWS Lambda (serverless)

When to use PaaS:

- You want to focus purely on writing application code
 - You don't want to manage OS patches, DB upgrades, or server configuration
 - You're building a new application from scratch
-

SaaS — Software as a Service

The entire application is managed by the provider. You just use it through a web browser or API.

Real-world analogy: Using a hotel room. Everything is managed — you just show up and use it.

AWS Examples: Amazon WorkMail (email), Amazon Chime (video conferencing), AWS Marketplace applications

When to use SaaS:

- Your team needs a productivity tool (email, collaboration)
 - You want zero infrastructure or maintenance responsibility
-

The Three Cloud Deployment Models

Model	Description	Example
Public Cloud	Resources owned and operated by AWS, shared among multiple customers	Standard AWS account
Private Cloud	Cloud infrastructure operated solely for one organisation	AWS Outposts, on-prem VMware
Hybrid Cloud	Combination of public and private cloud connected together	AWS + on-prem data centre via Direct Connect

Exam tip: For SAA-C03, the vast majority of scenarios involve **Public Cloud** architecture. Hybrid scenarios (Direct Connect, VPN) appear in Phase 7 of the roadmap.

Quick Reference: IaaS vs. PaaS vs. SaaS

Responsibility	IaaS	PaaS	SaaS
Physical hardware	AWS	AWS	AWS
Virtualisation	AWS	AWS	AWS
Operating System	You	AWS	AWS
Middleware/Runtime	You	AWS	AWS
Application Code	You	You	AWS
Data	You	You	AWS

1.2 AWS Global Infrastructure

Why Does Infrastructure Placement Matter?

When you deploy an application on AWS, where your resources physically sit determines:

- **Latency** — how fast your users get a response
- **Resilience** — whether a power outage in one city takes down your app
- **Compliance** — whether your data remains in the country it must legally stay in
- **Cost** — data transfer prices differ between regions

Regions

A **Region** is a **separate geographic area** where AWS has clustered multiple data centres. Each Region is completely **independent** from all other Regions — a failure in one Region cannot affect another.

Key facts:

- As of 2025, AWS has **35+ Regions** worldwide (growing constantly)
- Each Region has a unique name and code, e.g., `us-east-1` (N. Virginia), `ap-south-1` (Mumbai), `eu-west-1` (Ireland)
- Not all AWS services are available in all Regions
- Data does **not** automatically replicate across Regions — you explicitly choose

How to choose a Region for your application:

1. **Compliance and data residency** — Legal requirements must come first. If your data must stay in India, choose `ap-south-1`.
2. **Latency to end users** — Choose the Region closest to most of your users.
3. **Service availability** — Some new AWS services launch in `us-east-1` first, then expand

globally.

4. **Cost** — Prices vary slightly between Regions. `us-east-1` is often the cheapest.

Real-world example: A Canadian healthcare company must ensure patient data stays in Canada due to PIPEDA regulations. They would deploy everything in `ca-central-1` (Canada), regardless of whether another Region might be cheaper.

Availability Zones (AZs)

Within each Region, AWS has **multiple Availability Zones (AZs)**. Each AZ is one or more physically **separate data centres** with:

- Independent power supply
- Independent cooling
- Independent networking
- Physical separation of several kilometres from other AZs

Key facts:

- Most Regions have **3 AZs** (minimum 2, some have up to 6)
- AZs within a Region are connected by **high-bandwidth, low-latency private fibre**
- AZ names map to codes like `us-east-1a`, `us-east-1b`, `us-east-1c`
- **AWS shuffles AZ letters per account** — your `us-east-1a` may be a different physical data centre than another account's `us-east-1a`

Why AZs matter for architecture:

If you put all your EC2 instances in `us-east-1a` only, and that data centre has a fire, your entire application goes down. If you spread instances across `us-east-1a`, `us-east-1b`, and `us-east-1c`, one data centre failure only affects one-third of your capacity and your application stays up.

This is the most fundamental resilience pattern in AWS: design for multi-AZ.

Local Zones

Local Zones are extensions of an AWS Region, located in **major metropolitan areas** that are far from any existing Region.

Purpose: Deliver single-digit millisecond latency to end users in cities where no full Region exists.

Example: AWS has a Local Zone in Los Angeles, California. The nearest Region is `us-west-2` (Oregon). An application needing <10ms latency for LA users would deploy compute in the LA Local Zone, while keeping databases and storage in the parent `us-west-2` Region.

Key facts:

- Not all services are available in Local Zones
 - You opt in to Local Zones explicitly in the console
 - Local Zones are connected to the parent Region via AWS's private network
-

Wavelength Zones

Wavelength Zones embed AWS compute and storage services at the **edge of telecommunications networks** (5G carrier networks).

Purpose: Enable applications that require ultra-low latency (<10ms) for mobile and connected devices using 5G.

Example: A real-time multiplayer mobile game or autonomous vehicle telemetry processing would benefit from Wavelength Zones because game/vehicle data does not need to travel beyond the telecom network to reach AWS compute.

When you'd use it:

- AR/VR streaming to mobile devices
- Real-time video analytics at 5G scale
- IoT edge processing

SAA-C03 exam tip: You don't need to implement Wavelength Zones in labs. Just know the use case: ultra-low latency for 5G applications.

Infrastructure Hierarchy Summary

```
AWS Cloud
├── Region (e.g., ap-south-1 – Mumbai)
│   ├── Availability Zone A (one or more data centres)
│   ├── Availability Zone B (one or more data centres)
│   └── Availability Zone C (one or more data centres)
│       └── (each AZ has its own power, cooling, networking)
└── Extensions:
    ├── Local Zones (metro areas, extend parent Region)
    └── Wavelength Zones (inside telecom 5G networks)
```

1.3 Edge Locations and Points of Presence (PoP)

What Are Edge Locations?

Edge Locations are AWS infrastructure nodes distributed in **hundreds of cities worldwide** — far more than the number of Regions. They are not full data centres running EC2 instances. They exist for one purpose: **getting content closer to end users**.

Services that use Edge Locations:

- **Amazon CloudFront** (Content Delivery Network / CDN) — caches static and dynamic content
- **Amazon Route 53** (DNS) — routes user queries from the nearest PoP
- **AWS Global Accelerator** — routes traffic via the AWS backbone from the nearest PoP

How it works — CloudFront example:

Your website is hosted in `us-east-1` (N. Virginia). A user in Mumbai requests your website image. Without CloudFront, the image travels from Virginia to Mumbai — ~150ms latency. With CloudFront, the image is cached at the Mumbai Edge Location after the first request. Subsequent Mumbai users get it from Mumbai — ~5ms latency.

Regional Edge Caches

Regional Edge Caches sit between your origin server and Edge Locations. They have a larger cache than individual Edge Locations, meaning content that is not popular enough to stay cached at every Edge Location can still be served regionally rather than from your origin.

User Request Flow (CloudFront):

User → Nearest Edge Location → (cache miss?) → Regional Edge Cache → (cache miss?) → Origin (S3/EC2)

PoP vs. AZ vs. Region — Know the Difference

Concept	Count	Purpose	Example Service
Region	~35	Full AWS services, data residency	EC2, RDS, S3
Availability Zone	~100+	Fault isolation within a Region	EC2 per AZ
Edge Location / PoP	400+	Content caching, DNS, low-latency global routing	CloudFront, Route 53
Local Zone	Select cities	Low-latency compute near users	EC2 in metro
Wavelength Zone	Telecom networks	5G ultra-low latency	EC2 for mobile

1.4 Global vs. Regional vs. AZ-Scoped Services

Understanding which services are global, regional, or AZ-specific is critical for designing resilient architectures and answering exam questions correctly.

Global Services

These services operate **across all Regions** from a single control plane. There is no Region to select when using them.

Service	Why It's Global
IAM	Identity management must be consistent everywhere
Route 53	DNS must be globally accessible
CloudFront	CDN is inherently global
AWS Organizations	Account management spans all Regions
AWS WAF (when attached to CloudFront)	Filters at the global edge

Exam tip: IAM users, groups, roles, and policies are all global. An IAM user created in `us-east-1` can also log in from `eu-west-1`. There is no "Region" concept for IAM itself.

Regional Services

These services are **scoped to a single Region**. You must select a Region when creating them. They do not automatically replicate across Regions.

Service	Regional Scope
EC2	Instances, AMIs (unless copied), Key Pairs
S3	Buckets (globally unique name, but data stored in one Region)
VPC	Each VPC exists in one Region
Lambda	Functions deployed per Region
RDS	Databases in a specific Region
SQS, SNS	Queues and topics per Region

AZ-Scoped Resources

Some resources are tied to a **specific Availability Zone** and cannot be used from another AZ without copying.

Resource	AZ-Scoped?
EBS Volumes	Yes — attached to EC2 in same AZ only
EC2 Instances	Yes — run in a specific AZ
Subnets	Yes — each subnet lives in one AZ
Snapshots	No — stored in S3 (regional), can be used to

Resource	AZ-Scoped?
	create volumes in any AZ

Architecture implication: If you run a database on EBS in `us-east-1a` and the entire AZ fails, your EBS volume is inaccessible. This is why RDS Multi-AZ uses synchronous replication to a standby in a different AZ.

1.5 AWS Shared Responsibility Model

The Core Concept

The Shared Responsibility Model defines **exactly who is responsible for what** when you use AWS. This is not just a compliance concept — it has direct implications for what you must configure, patch, and monitor.

AWS is responsible for: "Security OF the Cloud" AWS physically secures its data centres, maintains the hypervisor, patches the underlying hardware, and guarantees the global infrastructure is available.

You are responsible for: "Security IN the Cloud" Everything you put on AWS — your operating systems, your application code, your data, your access control — is your responsibility.

Visual Breakdown by Service Type

EC2 (IaaS)

Layer	Responsible Party
Physical data centre	AWS
Hypervisor (Xen/Nitro)	AWS
Guest Operating System	You
Security patches for OS	You
Application code	You
Firewall (Security Group) configuration	You
Data encryption on EBS	You
IAM permissions to the instance	You

RDS (PaaS/Managed Service)

Layer	Responsible Party
Physical hardware	AWS
Database engine (MySQL, PostgreSQL, etc.)	AWS

Layer	Responsible Party
OS patches under the DB engine	AWS
Automated backups	AWS
Database schema design	You
Database user permissions	You
Enabling encryption at rest	You
Network access (Security Groups)	You

S3 (SaaS-like object storage)

Layer	Responsible Party
Physical storage hardware	AWS
Data durability (11 nines)	AWS
S3 service availability	AWS
Bucket policy and access control	You
Encryption of objects	You
Data classification and access management	You

Key Exam Scenarios

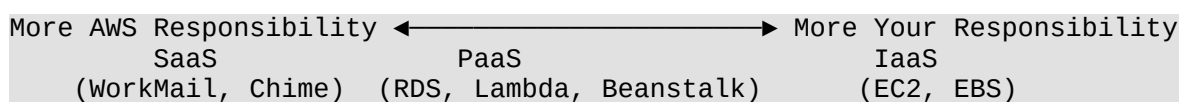
Scenario 1: An EC2 instance was compromised because it was running an outdated version of Apache with a known vulnerability. Whose responsibility? **Answer:** Yours. You manage the OS and application software on EC2.

Scenario 2: AWS has a hardware failure in a data centre. Some EBS volumes are corrupted. Whose responsibility? **Answer:** AWS's. Physical infrastructure is within AWS's scope. (Note: This is exactly why AWS offers EBS Snapshots — you should still take backups as a best practice.)

Scenario 3: An S3 bucket containing customer PII was left publicly accessible. Whose responsibility? **Answer:** Yours. Configuring bucket policies, ACLs, and Block Public Access settings is the customer's responsibility.

Shared Responsibility for Managed Services

As AWS manages more of the stack (moving from IaaS → PaaS → SaaS), the portion of responsibility that shifts to AWS increases. This is a key reason organisations adopt managed services — they offload operational burden to AWS.



1.6 AWS Service Health Dashboard & Personal Health Dashboard

AWS Service Health Dashboard

URL: health.aws.amazon.com

The **Service Health Dashboard** shows the **real-time and historical status** of every AWS service in every Region. It is a public-facing page — anyone can view it, no AWS account required.

What it shows:

- Current outages or service disruptions by service and Region
- Historical incidents (useful for understanding patterns)
- Status indicators: green (operational), yellow (degraded), red (service disruption)

When to use it:

- Your EC2 instances in `us-east-1` seem slow and you suspect an AWS-side issue
 - You want to verify an AWS service is healthy before troubleshooting your own code
 - Post-incident review to understand what AWS experienced
-

AWS Personal Health Dashboard (PHD)

Location: AWS Console → Services → Personal Health Dashboard (or search "Health")

The **Personal Health Dashboard** is **specific to your AWS account**. It surfaces events that **specifically affect resources in your account** — not general AWS-wide events.

What it shows:

- Scheduled maintenance windows that affect your running instances (e.g., EC2 host retirement)
- Performance degradation events affecting resources in your account
- Notifications about upcoming deprecations (e.g., retiring an old AMI)
- Recommendations for improving resilience specific to your setup

Real-world example: AWS needs to retire the physical host your EC2 instance is running on. The Service Health Dashboard shows nothing unusual (it's a routine operation). But your Personal Health Dashboard shows: "Your instance `i-0abc123` is scheduled for retirement on [date]. Please stop/start your instance before this date."

Key Difference for Exam

Feature	Service Health Dashboard	Personal Health Dashboard
Scope	All AWS customers	Your specific account

Feature	Service Health Dashboard	Personal Health Dashboard
Access	Public, no login needed	AWS Console login required
Content	AWS-wide outages	Events impacting your resources
Use case	General AWS status check	Account-specific alerts and actions

1.7 AWS Well-Architected Framework — Six Pillars Overview

What Is the Well-Architected Framework?

The **AWS Well-Architected Framework** is a set of **architectural best practices** developed by AWS Solutions Architects based on thousands of customer architecture reviews. It provides a consistent way to evaluate architectures and implement designs that will scale, remain secure, and keep costs in check.

The framework consists of **six pillars**. Each pillar has design principles, questions, and best practices. For SAA-C03, you need to understand what each pillar addresses and which pillar an architectural decision falls under.

Pillar 1 — Operational Excellence

"Run and monitor systems to deliver business value and continually improve processes."

Key practices:

- Use Infrastructure as Code (CloudFormation, CDK) — never manually configure
- Make small, frequent, reversible changes — avoid big-bang deployments
- Anticipate failure — conduct game days (simulate failures in production)
- Learn from all operational events — post-incident reviews

AWS tools: CloudFormation, Systems Manager, CloudWatch, X-Ray

Exam indicator phrase: "Which option reduces operational overhead?" → managed service over self-managed.

Pillar 2 — Security

"Protect information, systems, and assets while delivering business value."

Key practices:

- Implement a strong identity foundation — use IAM with least privilege
- Enable traceability — log everything (CloudTrail, CloudWatch Logs)

- Apply security at all layers — not just the perimeter
- Protect data in transit and at rest — TLS + KMS encryption
- Prepare for security events — have an incident response plan

AWS tools: IAM, KMS, CloudTrail, GuardDuty, Security Hub, WAF, Shield

Pillar 3 — Reliability

"Ensure a workload performs its intended function correctly and consistently."

Key practices:

- Automatically recover from failure — use ASG, Multi-AZ, Route 53 failover
- Test recovery procedures — don't assume backups work until you test them
- Scale horizontally — add more small resources rather than one big resource
- Stop guessing capacity — use Auto Scaling

AWS tools: Route 53, ASG, ELB, RDS Multi-AZ, AWS Backup, AWS FIS

Exam indicator phrase: "Which option provides the highest availability?" → Multi-AZ, ASG with health checks.

Pillar 4 — Performance Efficiency

"Use computing resources efficiently to meet system requirements."

Key practices:

- Use serverless architectures — no capacity planning needed (Lambda, Fargate)
- Go global in minutes — deploy in multiple Regions using CloudFormation
- Use the right database for the workload — DynamoDB for key-value, RDS for relational
- Experiment more often — cloud enables you to try new instance types cheaply

AWS tools: Lambda, EC2 Auto Scaling, ElastiCache, CloudFront, Compute Optimizer

Pillar 5 — Cost Optimisation

"Run systems to deliver business value at the lowest price point."

Key practices:

- Implement cloud financial management — use tagging and cost allocation
- Adopt a consumption model — pay only for what you use (On-Demand, Lambda)
- Measure overall efficiency — use Compute Optimizer for rightsizing

- Stop spending money on undifferentiated heavy lifting — use managed services
- Analyse and attribute expenditure — Cost Explorer, Budgets, Cost Anomaly Detection

AWS tools: Cost Explorer, Budgets, Savings Plans, Reserved Instances, Spot Instances, Compute Optimizer

Pillar 6 — Sustainability

"Minimise the environmental impact of running cloud workloads."

Key practices:

- Understand your impact — measure the carbon footprint of your workloads
- Establish sustainability goals — target reduced resource usage over time
- Maximise utilisation — right-size resources, avoid over-provisioning
- Adopt managed and serverless services — AWS data centres are more efficient than most on-premises

AWS tools: AWS Customer Carbon Footprint Tool, Compute Optimizer, Graviton processors (ARM-based, more energy efficient)

Six Pillars Quick Reference

Pillar	Core Question	Key AWS Tool
Operational Excellence	Can we operate and improve this?	CloudFormation, CloudWatch
Security	Is it protected?	IAM, KMS, GuardDuty
Reliability	Will it keep working?	Multi-AZ, ASG, Route 53
Performance Efficiency	Is it using resources wisely?	Lambda, ElastiCache, CloudFront
Cost Optimisation	Is it affordable?	Savings Plans, Compute Optimizer
Sustainability	Is it environmentally responsible?	Graviton, managed services

Chapter 1 Labs

Lab 1.1 — Explore the AWS Management Console and Identify Global vs. Regional Services

Estimated Time: 20 minutes

AWS Free Tier: Yes — no resources created, console browsing only

Objective

Familiarise yourself with the AWS Management Console and understand how the Region selector works.

Step-by-Step Instructions

Step 1: Log in to the AWS Console

1. Open console.aws.amazon.com
2. Log in with your AWS account credentials (create a free tier account if you don't have one)
3. After login, you land on the **AWS Management Console home page**

Step 2: Understand the Region Selector

1. Look at the **top-right corner** of the navigation bar — you'll see a Region name (e.g., "N. Virginia" or "Mumbai")
2. Click on it to open the **Region dropdown**
3. Observe the list of all available AWS Regions, grouped by geography
4. Select **Asia Pacific (Mumbai) ap-south-1**

Step 3: Observe a Regional Service

1. In the search bar at the top, type **EC2** and press Enter
2. Navigate to EC2 Dashboard
3. Notice at the top it says "EC2 Dashboard" with the Mumbai region
4. Now **change the Region** to **US East (N. Virginia) us-east-1** using the top-right dropdown
5. The EC2 dashboard updates — any instances you'd have here would only be in Virginia, not Mumbai
6. **Conclusion:** EC2 is a Regional service

Step 4: Observe a Global Service

1. In the search bar, type **IAM** and press Enter
2. Navigate to the IAM Dashboard
3. Notice: **the Region dropdown is greyed out** and shows "Global"
4. IAM works the same no matter which Region you're in
5. **Conclusion:** IAM is a Global service

Step 5: Find Other Global Services Try the same steps for these services and note whether they

show a Region or "Global":

- Route 53 (DNS) → Global
- CloudFront → Global
- S3 (special case — globally unique names, but data stored regionally — notice it shows "Global" in console)
- Lambda → Regional (has Region selector)

Step 6: Check Available Regions for a Service

1. Go to the [AWS Regional Services List](#)
 2. Look up a service — notice which services are available in every Region and which are only in select Regions
-

Lab 1.2 — Navigate the AWS Global Infrastructure Map

Estimated Time: 15 minutes

AWS Free Tier: Yes — online map, no console required

Objective

Visualise AWS Regions and AZs on the global map and understand geographic distribution.

Step-by-Step Instructions

Step 1: Open the Infrastructure Map

1. Navigate to aws.amazon.com/about-aws/global-infrastructure
2. Click on "Explore the infrastructure" or the interactive map

Step 2: Identify Regions

1. Each **cluster of dots** on the map represents a Region
2. Hover over a Region — you'll see its Region code (e.g., `ap-south-1`) and the number of AZs
3. Count the total number of available Regions

Step 3: Look at AZ Information

1. Select **Asia Pacific (Mumbai)** on the map
2. It shows 3 Availability Zones
3. Note how they are physically spread across the Mumbai metropolitan area

Step 4: Find the Newest Regions

1. Check which Regions are marked as "Planned" or "Coming Soon" — these represent AWS's expansion
2. Note why local data residency laws (e.g., in UAE, Spain, New Zealand) drive Region

creation

Step 5: Locate Edge Locations

1. On the same page, look for the **Edge Locations / PoPs** layer
2. Toggle it on — notice there are far more edge locations than Regions
3. Find edge locations in cities with no full Region (e.g., in many parts of Africa, South America)

Lab 1.3 — Review the Shared Responsibility Model for EC2 and S3

Estimated Time: 20 minutes

AWS Free Tier: Yes — documentation review only

Objective

Correctly categorise security responsibilities for common AWS services.

Exercise

For each item below, decide whether it is **AWS's responsibility** or **Your responsibility**. Write your answers, then check against the answer key.

#	Item	Service	Responsible Party
1	Physical server hardware maintenance	EC2	?
2	Installing OS security patches on the instance	EC2	?
3	Configuring Security Group rules	EC2	?
4	Ensuring EBS data is encrypted at rest	EC2/EBS	?
5	Hypervisor security	EC2	?
6	S3 bucket policy configuration	S3	?
7	S3 infrastructure uptime (99.999999999% durability)	S3	?
8	Enabling S3 Versioning	S3	?
9	RDS database engine patching	RDS	?
10	RDS database user and schema permissions	RDS	?

Answer Key:

1. AWS | 2. You | 3. You | 4. You | 5. AWS | 6. You | 7. AWS | 8. You | 9. AWS | 10. You
-

Lab 1.4 — Access the Personal Health Dashboard and Set Up Service Health Alerts

Estimated Time: 15 minutes

AWS Free Tier: Yes

Objective

Learn to monitor AWS service health for your account and set up notifications.

Step-by-Step Instructions

Step 1: Access the Personal Health Dashboard

1. Log into the AWS Console
2. In the top navigation bar, click the **bell icon** () → then click **"View all events"** or search for **"Health"** in the search bar
3. You'll land on **AWS Health Dashboard**

Step 2: Review Current Events

1. Under **"Your account health"**, look for any active events or notifications
2. If there are none, that's fine — your infrastructure is healthy
3. Click on **"AWS Health overview"** to see the global service status

Step 3: Navigate to Event Log

1. Click on **"Event log"** in the left panel
2. This shows all health events relevant to your account over time

Step 4: Set Up an Alert via CloudWatch (Optional but Recommended)

1. Go to **CloudWatch** in the console
2. Navigate to **Alarms** → **Create Alarm**
3. Select **AWS Health Events** as the data source
4. Configure it to send an SNS notification to your email when any health event occurs

Note: This creates a CloudWatch Alarm and an SNS topic. Both are free tier eligible. However, SMS notifications from SNS have a small cost (~\$0.001 per message). Use email notifications to stay within Free Tier.

Step 5: Check the Public Dashboard

1. Open a new browser tab (or incognito mode — no login needed)
2. Navigate to health.aws.amazon.com

3. See the real-time status of all AWS services globally
 4. Compare what you see here vs. what your Personal Health Dashboard showed
-

Chapter 1 Exam & Interview Questions

Multiple Choice — Certification Style

Q1. A company is deploying a web application and needs to ensure it remains available even if an entire AWS data centre fails. What should the solutions architect recommend?

- A) Deploy EC2 instances in multiple Regions
- B) Deploy EC2 instances in multiple Availability Zones within the same Region
- C) Use EC2 Reserved Instances for guaranteed capacity
- D) Enable Enhanced Networking on all EC2 instances

Answer: B

Explanation: AZ-level failures are more common than Region-level failures. Multi-AZ within a Region provides fault tolerance against data centre failures. Multi-Region is needed for Region-level resilience but is more complex and expensive.

Q2. Which of the following is a customer's responsibility under the AWS Shared Responsibility Model when using Amazon EC2?

- A) Patching the physical hardware
- B) Securing the hypervisor layer
- C) Installing and maintaining the guest operating system
- D) Maintaining the network infrastructure within the data centre

Answer: C

Explanation: EC2 is IaaS. AWS manages the physical hardware, network, and hypervisor. You manage the OS, applications, and data.

Q3. A company has regulatory requirements that its data must remain within a specific country. Which AWS concept directly addresses this requirement?

- A) Availability Zones
- B) Edge Locations
- C) Regions
- D) Local Zones

Answer: C

Explanation: Data in an AWS Region does not automatically replicate to other Regions. Choosing the correct Region is how you satisfy data residency requirements.

Q4. What is the primary purpose of AWS Edge Locations?

- A) Running EC2 instances closer to end users
- B) Caching content and performing DNS resolution close to users
- C) Providing dedicated physical hardware for large enterprises
- D) Creating isolated fault domains within a Region

Answer: B

Explanation: Edge Locations are used by CloudFront (CDN caching) and Route 53 (DNS). They do not run EC2 instances.

Q5. An application running on EC2 was compromised due to an unpatched vulnerability in the Linux kernel. According to the Shared Responsibility Model, who is responsible for this?

- A) AWS, because they manage the underlying infrastructure
- B) The customer, because OS-level patching is the customer's responsibility on EC2
- C) Shared responsibility — both AWS and the customer are equally responsible
- D) AWS, because the Linux kernel is open source and not the customer's code

Answer: B

Explanation: For EC2 (IaaS), operating system patching is entirely the customer's responsibility. AWS patches the hypervisor, not the guest OS.

Q6. Which of the following AWS services are GLOBAL in scope? (Select TWO)

- A) EC2
- B) IAM
- C) S3 Buckets
- D) Route 53
- E) Lambda

Answer: B and D

Explanation: IAM and Route 53 are global services. EC2 and Lambda are regional. S3 bucket names are globally unique but buckets are stored in a specific Region.

Q7. A company wants to receive notifications when AWS scheduled maintenance will affect their EC2 instances. Which service should they use?

- A) AWS Service Health Dashboard
- B) AWS Personal Health Dashboard
- C) Amazon CloudWatch
- D) AWS Config

Answer: B

Explanation: The Personal Health Dashboard surfaces events that specifically affect resources in your AWS account, including instance-level maintenance notifications.

Q8. Under the Well-Architected Framework, which pillar focuses on the ability of a workload to

recover from infrastructure failures and scale to meet demand?

- A) Performance Efficiency
- B) Operational Excellence
- C) Reliability
- D) Cost Optimisation

Answer: C

Explanation: The Reliability pillar covers automatic recovery from failures, horizontal scaling, and testing recovery procedures.

Short Answer — Interview Style

Q9. What is the difference between an Availability Zone and a Region?

Answer: A Region is a geographically separate area of the world (like Mumbai or N. Virginia) containing multiple Availability Zones. An AZ is one or more physically separate data centres within that Region with independent power, cooling, and networking. Regions provide geographic and jurisdictional separation; AZs provide fault isolation within the same geography.

Q10. Explain the Shared Responsibility Model in your own words. Give one example.

Answer: AWS is responsible for the security "of" the cloud — the physical infrastructure, hardware, and hypervisor. Customers are responsible for security "in" the cloud — their data, applications, OS configuration, and access control settings. Example: If I use EC2, AWS secures the physical server and hypervisor. I must patch the Linux OS, configure Security Group rules, and encrypt my EBS volumes. If any of those are misconfigured and my instance is compromised, that is my responsibility, not AWS's.

Q11. A new developer on your team says, "Since we're using a managed database (RDS), we don't need to worry about any security for it." Is this correct? What would you tell them?

Answer: That is incorrect. While RDS does shift significant responsibility to AWS (OS patching, engine patching, automated backups), several critical security responsibilities remain with the customer: configuring database security groups (network access), setting up IAM policies for who can access the RDS API, managing database user credentials and permissions within the database, enabling encryption at rest (KMS), and enforcing encryption in transit (SSL/TLS). Choosing a managed service reduces operational burden but does not eliminate security responsibility.

Q12. What is the difference between the AWS Service Health Dashboard and the Personal Health Dashboard?

Answer: The Service Health Dashboard is a public page showing the real-time status of all AWS services for all customers globally. It shows broad outages. The Personal Health Dashboard is account-specific — it shows events that directly impact resources in your particular AWS account, such as a scheduled EC2 host retirement or performance degradation on your specific instances.

The Personal Health Dashboard is the actionable one — it tells you what you need to do.

Chapter 2 — IAM Essentials

Overview

Identity and Access Management (IAM) is the foundation of security in AWS. Before any resource in your account can be accessed — whether by a person, an application, or another AWS service — IAM decides whether that access is allowed.

IAM is also one of the **most heavily tested topics on the SAA-C03 exam**. Understanding it thoroughly will give you a significant advantage across almost every domain in the exam, because nearly every scenario involves some aspect of permissions.

2.1 IAM Core Concepts

The Problem IAM Solves

When you create an AWS account, you get a **root user** — a single identity with unlimited access to everything in the account. The root user is like the master key to your entire cloud infrastructure. If someone gets access to it, they can delete everything, run up massive bills, or steal all your data.

IAM solves this by allowing you to create **separate identities with specific, limited permissions** so that:

- Different people get access to only the services they need
 - Applications can call AWS APIs using specific credentials
 - AWS services can interact with each other in controlled ways
-

IAM Users

An **IAM User** is an identity that represents a **specific person or application** that interacts with AWS. It has permanent credentials: a username/password for the console and/or access keys for programmatic access.

Key characteristics:

- Belongs to exactly one AWS account
- Can have both console access (password) and programmatic access (access keys)
- Has NO permissions by default — you must explicitly grant them
- Has a unique ARN identifier

When to create IAM Users:

- A human employee needs long-term access to your AWS account
- A legacy application needs static AWS credentials (not recommended for new applications — use roles instead)

Best practice: Do not create IAM users for applications running on AWS services (EC2, Lambda). Use **IAM Roles** instead. Access keys can be accidentally committed to code repositories.

IAM Groups

An **IAM Group** is a collection of IAM Users. Groups make permission management easier at scale.

Key characteristics:

- A group can contain multiple users
- A user can belong to multiple groups (maximum 10 groups per user)
- Groups **cannot** contain other groups (no nesting)
- Groups have no credentials — you cannot log in as a group
- Permissions attached to a group are inherited by all users in that group

Real-world example:

Imagine you have a DevOps team of 8 engineers. Instead of attaching an `AdministratorAccess` policy to each of the 8 users individually, you:

1. Create a group called `DevOps`
2. Attach `AdministratorAccess` to the group
3. Add all 8 engineers as members

When a new engineer joins, add them to the group — done. When someone leaves, remove them from the group — all their group-based permissions are instantly revoked.

IAM Roles

An **IAM Role** is an identity with specific permissions that can be **temporarily assumed** by any trusted principal — a user, a service, or even an account.

Key differences from IAM Users:

- No permanent credentials — roles issue **temporary security tokens** via STS
- Can be assumed by multiple entities
- Can be assumed by AWS services (like EC2 or Lambda) — no need to embed credentials in code

Common Role Use Cases:

Use Case	Description
EC2 Instance Role	Allows EC2 to call S3 or DynamoDB without storing access keys in code
Lambda Execution Role	Allows Lambda function to write logs to CloudWatch
Cross-Account Role	Allows a user in Account A to access resources in Account B
Federated Identity Role	Allows a corporate user authenticated via SAML to get AWS access

How Role Assumption Works:

1. EC2 instance wants to read from S3
2. EC2 calls STS (Security Token Service): "Give me credentials for this role"
3. STS checks the role's trust policy: "Does EC2 service have permission to assume this role?"
4. Yes → STS issues temporary credentials (AccessKeyId, SecretAccessKey, SessionToken)
5. These credentials expire automatically (1-12 hours by default)
6. EC2 uses credentials to call S3 API

This is far more secure than embedding static access keys — if credentials expire, they are useless. If a bad actor gets them from a running instance, they'll expire soon.

Users vs. Groups vs. Roles — Summary

Feature	User	Group	Role
Has credentials	Yes (permanent)	No	No (temporary via STS)
Represents	A person or app	A collection of users	An identity to be assumed
Can log in to console	Yes	No	No (assumed, not logged into)
Can have policies attached	Yes	Yes	Yes
Can be assumed by AWS services	No	No	Yes

2.2 IAM Policies

What Is an IAM Policy?

An **IAM Policy** is a JSON document that defines **permissions**. It specifies:

- **What actions** are allowed or denied (e.g., `s3:GetObject`)

- **On what resources** (e.g., a specific S3 bucket)
- **Under what conditions** (e.g., only from a specific IP address)

Policies by themselves do nothing — they must be **attached** to a User, Group, or Role to take effect.

Policy Structure

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowsS3Read",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3::my-company-bucket",
        "arn:aws:s3::my-company-bucket/*"
      ],
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": "203.0.113.0/24"
        }
      }
    }
  ]
}
```

Breaking down each element:

Element	Required	Description
Version	Recommended	Policy language version. Always use "2012-10-17"
Statement	Yes	Array of one or more permission statements
Sid	No	Statement ID — a label for your reference
Effect	Yes	"Allow" or "Deny"
Action	Yes	AWS API operation(s). Format: service:Operation
Resource	Yes (for most)	The ARN(s) the action applies to. "*" means all resources
Condition	No	Optional conditions under which the statement applies

Understanding the Effect: Allow vs. Deny

AWS uses a default-deny model. By default, all access is denied. Policies must explicitly allow actions.

IAM Evaluation Logic:

1. Start with **implicit deny** (nothing is allowed by default)
2. Check all applicable policies
3. If there is an explicit **Deny** → deny immediately, no override possible
4. If there is an explicit **Allow** → allow the action
5. If neither → deny (implicit deny)

Critical rule: An explicit **Deny** **always overrides any Allow**. This is how organisations enforce guardrails — an SCP (Service Control Policy) with an explicit Deny cannot be overridden by any IAM Allow policy.

Example scenario:

- Policy A says: `Allow s3:*` on all resources
- Policy B says: `Deny s3:DeleteObject` on all resources
- Both are attached to the same user

Result: The user can do all S3 operations EXCEPT DeleteObject. The Deny wins.

Types of IAM Policies

1. AWS Managed Policies

Pre-built policies created and maintained by AWS. They cover common use cases.

Examples:

- `AdministratorAccess` — full access to all AWS services
- `AmazonS3ReadOnlyAccess` — read-only access to all S3 buckets
- `AmazonEC2FullAccess` — full access to EC2

Pros: Ready to use, AWS keeps them updated when services add new features

Cons: Often broader than necessary — may violate least privilege

2. Customer Managed Policies

Policies you create and maintain in your account. They can be attached to multiple users/roles.

When to use: When AWS managed policies are too broad or don't fit your exact needs.

Example: A policy that allows reading only from `my-company-logs` bucket, not all S3 buckets.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject", "s3:ListBucket"],
      "Resource": [
        "arn:aws:s3::my-company-logs",
        "arn:aws:s3::my-company-logs/*"
      ]
    }
  ]
}
```

3. Inline Policies

Policies embedded directly into a single user, group, or role. They do not exist independently.

When to use: When a permission should only ever apply to one specific entity and you want a strict 1:1 binding.

Cons: Hard to manage at scale. If you delete the user, the policy is also deleted. Not reusable.

Best practice for SAA-C03 exam answers: Prefer **Customer Managed Policies** over inline policies for scalability and reusability.

Identity-Based vs. Resource-Based Policies

Identity-Based Policies

Attached to an **IAM identity** (user, group, role). They define what that identity can do.

Example: "This IAM role is allowed to read S3 objects."

You can use:

- AWS Managed Policies
 - Customer Managed Policies
 - Inline Policies
-

Resource-Based Policies

Attached directly to an **AWS resource** (like an S3 bucket, SQS queue, or KMS key). They define who can access that resource.

Key difference: Resource-based policies can grant access to **principals in other AWS accounts** without requiring role assumption.

Example — S3 Bucket Policy:

```
{
  "Version": "2012-10-17",
```

```
"Statement": [
  {
    "Effect": "Allow",
    "Principal": {
      "AWS": "arn:aws:iam::123456789012:user/Shaad"
    },
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::my-bucket/*"
  }
]
```

This bucket policy says: "Allow the IAM user 'Shaad' from account 123456789012 to download objects from this bucket."

Services that support resource-based policies:

- S3 (bucket policy)
- SQS (queue policy)
- SNS (topic policy)
- Lambda (resource-based policy)
- KMS (key policy)
- API Gateway (resource policy)

How Access is Evaluated — The Full Picture

When a request is made, AWS evaluates policies in this order:

1. Is this the root user? → Full access (avoid using root)
2. Are there any SCP (Organisation policy) Denies? → Deny immediately
3. Are there any Resource-Based Policies? → Evaluate
4. Are there any Identity-Based Policies? → Evaluate
5. Are there any Permission Boundaries? → Check upper limit
6. Are there any Session Policies? → Apply

If at any step there is an explicit **Deny** → the action is denied. If there is no explicit Allow anywhere → **implicit deny**.

2.3 Amazon Resource Names (ARNs)

What Is an ARN?

An **Amazon Resource Name (ARN)** is a unique identifier for any resource in AWS. ARNs are used in IAM policies to specify exactly which resources a policy applies to.

ARN Format:

```
arn:partition:service:region:account-id:resource-type/resource-id
```

Part	Description	Example
arn	Always "arn"	arn
partition	AWS partition	aws (standard), aws-cn (China), aws-us-gov (GovCloud)
service	AWS service	s3, ec2, iam, lambda
region	Region code	ap-south-1, us-east-1 (empty for global services like IAM)
account-id	12-digit account ID	123456789012 (empty for S3 bucket ARNs)
resource	Resource identifier	bucket name, user name, instance ID

ARN Examples

```
# IAM User (global – no region or account in some parts)
arn:aws:iam::123456789012:user/Shaad

# IAM Role
arn:aws:iam::123456789012:role/MyEC2Role

# S3 Bucket (no region, no account-id in bucket ARN)
arn:aws:s3::my-company-bucket

# S3 Object (using wildcard to match all objects in bucket)
arn:aws:s3::my-company-bucket/*

# EC2 Instance
arn:aws:ec2:ap-south-1:123456789012:instance/i-0abc1234def56789

# Lambda Function
arn:aws:lambda:us-east-1:123456789012:function:my-function

# DynamoDB Table
arn:aws:dynamodb:us-east-1:123456789012:table/Users
```

ARN Wildcards

In IAM policy Resource fields, wildcards make ARNs flexible:

Wildcard	Meaning	Example
*	Zero or more characters	arn:aws:s3:::* = all S3 buckets
?	Any single character	Rarely used

Common pattern — grant access to all objects in a bucket:

```
"Resource": [  
  "arn:aws:s3::my-bucket",      ← the bucket itself (needed for ListBucket)  
  "arn:aws:s3::my-bucket/*"    ← all objects inside the bucket  
]
```

Important: S3 `ListBucket` requires permission on the bucket ARN itself. S3 `GetObject` and `PutObject` require permission on `bucket/*` (the objects). This is a common exam question.

2.4 Multi-Factor Authentication (MFA)

What Is MFA?

Multi-Factor Authentication adds a **second layer of verification** beyond just a password. Even if an attacker steals your password, they cannot log in without also having the second factor.

IAM supports three categories of MFA:

1. Virtual MFA Devices

Software apps on your phone or computer that generate time-based one-time passwords (TOTP) that change every 30 seconds.

Compatible apps: Google Authenticator, Authy, Microsoft Authenticator, 1Password

How it works:

1. You set up the app by scanning a QR code from the AWS Console
2. Every time you log in, after entering your password, AWS asks for a 6-digit code
3. You open the app — it shows a 6-digit code valid for 30 seconds
4. Enter the code → access granted

Cost: Free (you use your existing phone)

Best for: Individual developers, small teams

2. U2F Security Keys (FIDO2)

Physical USB or NFC hardware tokens (like YubiKey). You plug it in and tap a button when prompted.

How it works:

1. Register the physical key with your AWS account
2. At login, after your password, plug in the key and tap the button
3. The key performs a cryptographic challenge-response — no code to type

Cost: \$25–\$60 for a YubiKey

Best for: High-security environments, privileged admin accounts

3. Hardware MFA Tokens (OTP Tokens)

Physical devices that display a 6-digit code — similar to virtual MFA but on dedicated hardware (Gemalto tokens).

Best for: Organisations that cannot use smartphones (regulated industries), or for the AWS root account at the hardware level

MFA for the Root Account

This is non-negotiable: enable MFA on the root account immediately after creating an AWS account.

The root account has unlimited power — it can close the account, cancel support plans, and delete everything. With just a strong password and MFA, the root account is extremely well protected.

IAM Best Practice: After enabling MFA on root, do not use the root account for everyday tasks. Create an IAM admin user for daily operations.

MFA in IAM Policies — Conditional MFA Enforcement

You can write IAM policies that **require MFA** for sensitive operations:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": "*",
      "Resource": "*",
      "Condition": {
        "BoolIfExists": {
          "aws:MultiFactorAuthPresent": "false"
        }
      }
    }
  ]
}
```

This policy denies all actions if the user has not authenticated with MFA. Attach this to a group — users must use MFA or they can't do anything.

2.5 IAM Best Practices

These best practices are also AWS exam content — you should know all of them.

1. Lock Away the Root Account

- Enable MFA on root immediately
 - Do not create access keys for root (if they exist, delete them)
 - Only use root for tasks that specifically require it (e.g., changing AWS account settings, enabling IAM access to the Billing console)
-

2. Create Individual IAM Users

- Never share credentials between people
 - Each person gets their own IAM user
 - This way, you can revoke one person's access without affecting others
 - Audit trails in CloudTrail show exactly which user took which action
-

3. Use Groups to Assign Permissions

- Assign permissions to groups, not individual users
 - Users inherit permissions from their groups
 - When roles change, update group membership — don't modify individual user policies
-

4. Follow the Principle of Least Privilege

Grant only the permissions required to do the job — nothing more.

- Don't give AdministratorAccess to a user who only needs to manage EC2
 - Start with minimal permissions and expand as needed
 - Regularly review and remove unused permissions
-

5. Use IAM Roles for Applications on AWS

- EC2 instances, Lambda functions, and ECS tasks should use **IAM Roles**, not access keys
 - Roles provide temporary, automatically-rotated credentials
 - No secret to store, no credential to accidentally expose in code
-

6. Use Roles for Cross-Account Access

- Never share IAM user credentials between accounts
- Create a role in the target account and allow the source account to assume it

- This creates an auditable, revocable access pattern
-

7. Rotate Credentials Regularly

- If you do have access keys (for legacy applications), rotate them regularly
 - IAM provides a credential report to identify users with old, unused keys
 - Enable automated key rotation where possible
-

8. Remove Unnecessary Credentials

- Identify users who haven't logged in for 90+ days using the IAM credential report
 - Disable or delete their credentials
 - Remove access keys that haven't been used recently
-

9. Enable MFA for Privileged Users

- Anyone with administrative permissions must have MFA enabled
 - Enforce this with IAM policy conditions (`aws:MultiFactorAuthPresent`)
-

10. Use IAM Access Analyser

- AWS IAM Access Analyser identifies resources in your account that are accessible from outside your account or organisation
 - Helps find overly permissive S3 bucket policies, IAM roles, etc.
-

2.6 IAM Password Policy, Account Alias & Credential Report

IAM Password Policy

The **IAM Password Policy** defines rules that all IAM user passwords must follow. You configure it at the account level.

Configurable settings:

- Minimum password length (8–128 characters)
- Require at least one uppercase letter
- Require at least one lowercase letter
- Require at least one number
- Require at least one non-alphanumeric character (symbol)

- Allow users to change their own passwords
- Prevent password reuse (remember last N passwords, up to 24)
- Password expiration (force users to change password every N days, up to 1095)
- Require administrator reset after expiration

Where to configure: Console → IAM → Account Settings → Password Policy

Note: The password policy applies to IAM user passwords only. It does NOT apply to the root account password.

AWS Account Alias

By default, your AWS account is identified by its **12-digit Account ID** (e.g., 123456789012). The IAM console sign-in URL would be:

```
https://123456789012.signin.aws.amazon.com/console
```

You can set a **friendly Account Alias** to make this more readable:

```
https://my-company-dev.signin.aws.amazon.com/console
```

Where to set: Console → IAM → Dashboard → Account Alias → Edit

Rules:

- Must be globally unique across all AWS accounts
- Only lowercase letters, numbers, and hyphens
- Cannot start or end with a hyphen

Practical use: When you have multiple AWS accounts (dev, staging, prod), a clear alias in the sign-in URL prevents users from accidentally logging into the wrong account.

IAM Credential Report

The **IAM Credential Report** is a downloadable CSV file listing all IAM users in your account and the status of their credentials.

What it shows for each user:

- User creation date
- Whether they have a console password
- When they last logged in with their password
- Whether MFA is enabled
- Whether they have access keys
- When each access key was last used

- Whether access keys are active or inactive

Where to generate: Console → IAM → Credential Report → Download Report

Use cases:

- Security audits — who has active access keys that haven't been used in 90 days?
 - Compliance — verify MFA is enabled for all users
 - Offboarding — verify a departed employee's credentials are fully disabled
 - Automation — use the API to generate and parse this report in security workflows
-

2.7 IAM Policy Simulator

What Is the Policy Simulator?

The **IAM Policy Simulator** is a tool that lets you **test what a user, group, or role can and cannot do** — without actually making real API calls.

URL: policysim.aws.amazon.com

Why Use It?

Debugging IAM permission errors can be frustrating. Instead of making an actual API call and seeing "Access Denied," you can:

1. Select a user, group, or role
 2. Choose an AWS service and action (e.g., `s3:GetObject`)
 3. Specify a resource ARN
 4. Click "Simulate" → see immediately if it would be Allowed or Denied and WHY
-

How to Use the Policy Simulator

Step 1: Navigate to the Policy Simulator (link above or IAM Console → Tools → Policy Simulator)

Step 2: In the left panel, select the entity to simulate:

- Select **IAM Users, Groups, and Roles**
- Choose a specific user or role from your account

Step 3: In the right panel, select the service to test:

- Example: Select **Amazon S3**
- Select actions: `s3:GetObject`, `s3:PutObject`, `s3:DeleteObject`

Step 4: Optionally specify a resource ARN (for resource-specific policies):

- Example: `arn:aws:s3:::my-bucket/*`

Step 5: Click "**Run Simulation**"

Reading the results:

- **Allowed** — The policy grants this action
- **Denied (Implicit)** — No policy allows this action (default deny)
- **Denied (Explicit)** — A policy explicitly denies this action

For any denied result, click on the action to see exactly which policy statement caused the denial.

Chapter 2 Labs

Lab 2.1 — Create IAM Users, Groups, and Assign Policies

Estimated Time: 30 minutes

AWS Free Tier: Yes — IAM is free

Note: You will create IAM users and groups. No resources that incur costs are created in this lab.

Objective

Create a realistic IAM structure for a small team: an S3 developer and an EC2 operator.

Step-by-Step Instructions

Step 1: Create an IAM Group for S3 Developers

1. Go to IAM Console → **Groups** → Create Group
2. Group name: `S3-Developers`
3. On the permissions page, search for `AmazonS3ReadOnlyAccess`
4. Check the box next to it → click **Create Group**

Step 2: Create an IAM Group for EC2 Operators

1. Create another group: `EC2-Operators`
2. Attach the policy: `AmazonEC2ReadOnlyAccess`
3. Click **Create Group**

Step 3: Create IAM User — Alice (S3 Developer)

1. Go to **Users** → **Create User**
2. Username: `alice`

3. Access type: Check **"Provide user access to the AWS Management Console"**
4. Console password: **Custom password** → set a strong password
5. Uncheck "User must create a new password at next sign-in" (for lab convenience only)
6. Click **Next**
7. On the permissions page, select **"Add user to group"**
8. Select the S3-Developers group
9. Click **Next** → **Create User**

Step 4: Create IAM User — Bob (EC2 Operator)

1. Repeat the process above for user bob
2. Add Bob to the EC2-Operators group

Step 5: Create and Attach a Custom Inline Policy Let's add an inline policy directly to Alice that denies S3 DeleteObject:

1. Go to **Users** → **alice** → **Permissions** → **Add permissions** → **Create inline policy**
2. Switch to **JSON** tab and paste:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Deny",
      "Action": "s3:DeleteObject",
      "Resource": "*"
    }
  ]
}
```

1. Policy name: DenyS3Delete
2. Click **Create policy**

Step 6: Test Login as Alice

1. Copy the IAM sign-in URL from IAM Dashboard
2. Open a new incognito browser window
3. Go to the sign-in URL
4. Log in as **alice** with the password you set
5. Navigate to S3 — you can browse buckets (S3ReadOnly allows this)
6. Try navigating to EC2 — you should see "You don't have permission to list EC2 instances"

Step 7: Test Login as Bob

1. In a second incognito window, log in as **bob**
2. Navigate to EC2 — you can view instances (EC2ReadOnly)

3. Navigate to S3 — access is denied (no S3 policy for Bob)

Step 8: Add Bob to Both Groups

1. Back in the main account, go to **Groups** → **S3-Developers** → **Users** → **Add Users**
 2. Add bob to the S3-Developers group as well
 3. Log back in as Bob — now Bob has BOTH EC2 and S3 read access
-

Lab 2.2 — Enable MFA on the Root Account and an IAM User Account

Estimated Time: 20 minutes

AWS Free Tier: Yes

Prerequisites: A smartphone with Google Authenticator or Authy installed

△ **Important:** Enabling MFA on root is a critical security step. If you lose your MFA device, recovery requires contacting AWS Support. Keep a backup of your MFA setup codes.

Step-by-Step Instructions

Part A: Enable MFA on the Root Account

1. Log into the AWS Console as the **root user** (using the email address you used to create the account)
2. In the top-right corner, click on your **account name** → **Security credentials**
3. Under **Multi-factor authentication (MFA)** → Click **Assign MFA device**
4. Select **Authenticator app** → Next
5. AWS shows a QR code
6. Open your authenticator app on your phone → Tap "+" or "Add account" → Scan the QR code
7. Enter the first 6-digit code from the app into MFA code 1
8. Wait 30 seconds for the code to refresh
9. Enter the new 6-digit code into MFA code 2
10. Click **Add MFA**
11. You'll see "MFA device assigned successfully"

Part B: Enable MFA on the Alice IAM User

1. Go to IAM → **Users** → **alice**
2. Click the **Security credentials** tab
3. Under "Multi-factor authentication (MFA)" → **Assign MFA device**
4. Device name: `alice-mfa`

5. Select **Authenticator app** → Next
6. Scan the QR code with a new entry in your authenticator app
7. Enter two consecutive codes → Click **Add MFA**

Verification:

1. Open an incognito browser → Sign in as Alice
 2. Enter username and password → AWS now asks for the MFA code
 3. Enter the 6-digit code from your authenticator app
 4. Successfully logged in with MFA
-

Lab 2.3 — Create an IAM Role for EC2 and Attach an S3 Read Policy

Estimated Time: 20 minutes

AWS Free Tier: Yes

Note: This lab creates an IAM Role only. If you launch an EC2 instance to fully test it, that may incur cost if you exceed Free Tier EC2 hours. The role creation itself is free. The lab walkthrough includes the role-only creation; EC2 launch is optional.

Objective

Create an IAM Role that allows an EC2 instance to read from S3, without storing any credentials on the instance.

Step-by-Step Instructions

Step 1: Create the IAM Role

1. IAM Console → **Roles** → **Create role**
2. Select **Trusted entity type: AWS service**
3. Use case: **EC2** (this is the trust policy — saying EC2 is allowed to assume this role)
4. Click **Next**

Step 2: Attach a Permission Policy

1. Search for **AmazonS3ReadOnlyAccess**
2. Check the box next to it
3. Click **Next**

Step 3: Name and Review

1. Role name: **EC2-S3ReadOnly-Role**
2. Description: "Allows EC2 instances to read from S3 buckets"
3. Review that the trusted entity shows **ec2.amazonaws.com**

4. Click **Create role**

Step 4: Inspect the Role's Trust Policy

1. Go to the newly created role
2. Click on the **Trust relationships** tab
3. You'll see the trust policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "ec2.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

This means: "The EC2 service is allowed to assume this role."

Step 5: (Optional — incurs EC2 cost if beyond Free Tier) Attach the Role to an EC2 Instance

1. Launch an EC2 instance
2. In the instance configuration, under **Advanced Details** → **IAM instance profile**, select **EC2-S3ReadOnly-Role**
3. SSH into the instance
4. Run: `aws s3 ls` (no credentials needed — the role provides them)
5. Run: `aws sts get-caller-identity` — it will show the role ARN, not an access key

Lab 2.4 — Use the IAM Policy Simulator to Validate Effective Permissions

Estimated Time: 20 minutes

AWS Free Tier: Yes

Objective

Use the Policy Simulator to test what the users created in Lab 2.1 can and cannot do.

Step-by-Step Instructions

Step 1: Open the Policy Simulator

1. Navigate to the IAM Console
2. In the left panel, click **Policy Simulator** (under Tools) or go to `polysim.aws.amazon.com`

Step 2: Simulate for User Alice

1. On the left, under **IAM Users, Groups, and Roles**, select your user `alice`
2. You'll see the policies applied to Alice:
 - `AmazonS3ReadOnlyAccess` (from the `S3-Developers` group)
 - `DenyS3Delete` (inline policy)

Step 3: Test S3 Actions

1. In the right panel, under **Policy Simulator**, click **Select service**
2. Choose **Amazon S3**
3. Select these actions: `GetObject`, `PutObject`, `DeleteObject`, `ListBucket`
4. Click **Run Simulation**

Expected Results:

- `s3:GetObject` → **Allowed** (`S3ReadOnly` permits this)
- `s3:ListBucket` → **Allowed** (`S3ReadOnly` permits this)
- `s3:PutObject` → **Denied (Implicit)** (`S3ReadOnly` doesn't grant write; no explicit deny either)
- `s3>DeleteObject` → **Denied (Explicit)** (the inline `DenyS3Delete` policy explicitly denies this)

Step 4: Test EC2 Actions for Alice

1. Change the service to **Amazon EC2**
2. Select `DescribeInstances`, `RunInstances`
3. Click **Run Simulation**

Expected Results:

- `ec2:DescribeInstances` → **Denied (Implicit)** — Alice has no EC2 permissions
- `ec2:RunInstances` → **Denied (Implicit)** — no EC2 permissions

Step 5: Simulate for User Bob (after adding to S3-Developers group)

1. Select user `bob`
2. Test both S3 and EC2 actions
3. Bob should have S3 read access AND EC2 read access (from being in both groups)

Step 6: Understand the Difference Between Implicit and Explicit Deny

1. For `s3:PutObject` on Alice: look at the simulation result — it says "Implicitly Denied" and shows no matching policy
2. For `s3>DeleteObject` on Alice: it says "Explicitly Denied" and shows the `DenyS3Delete` inline policy as the reason

3. This is the difference: an explicit Deny cannot be overridden; an implicit deny means "no allow was found"
-

Chapter 2 Exam & Interview Questions

Multiple Choice — Certification Style

Q1. A company wants to grant temporary AWS access to an external auditing firm so they can review CloudTrail logs in S3. The auditing firm has their own AWS account. What is the MOST secure way to provide this access?

- A) Create an IAM user for each auditor and share the credentials
- B) Create a cross-account IAM Role and allow the auditing firm's account to assume it
- C) Share the AWS root account credentials with the auditing firm
- D) Make the S3 bucket containing CloudTrail logs public

Answer: B

Explanation: Cross-account IAM Roles are the correct way to grant temporary, auditable access to another AWS account. The role provides temporary credentials (via STS), access can be revoked instantly, and all actions are logged in CloudTrail.

Q2. An application running on EC2 needs to read objects from an S3 bucket. What is the MOST secure way to configure this?

- A) Store access keys as environment variables in the EC2 instance
- B) Hardcode access keys in the application source code
- C) Create an IAM Role with S3 read permissions and attach it to the EC2 instance
- D) Create an IAM user, generate access keys, and store them in the instance user data script

Answer: C

Explanation: IAM Roles attached to EC2 provide temporary, automatically-rotated credentials via the Instance Metadata Service. Access keys stored anywhere (env variables, code, user data) are a security risk.

Q3. A developer is denied access to `s3:DeleteObject` despite having an `AmazonS3FullAccess` policy attached. Which of the following is the MOST likely reason?

- A) S3 Full Access does not include DeleteObject
- B) An explicit Deny policy is attached somewhere in their permission chain
- C) The S3 bucket is in a different Region
- D) The developer needs to re-authenticate to pick up the policy

Answer: B

Explanation: An explicit Deny always overrides any Allow in AWS IAM. If AmazonS3FullAccess allows DeleteObject but another policy (inline, SCP, permission boundary) explicitly denies it, the deny wins.

Q4. Which of the following statements about IAM Groups is CORRECT?

- A) IAM Groups can be nested — you can add a group as a member of another group
- B) IAM Groups can be used to log in to the AWS Management Console
- C) IAM Groups are a way to apply permissions to multiple users simultaneously
- D) IAM Groups can assume IAM Roles

Answer: C

Explanation: Groups cannot be nested, cannot log in, and cannot assume roles. Their sole purpose is to simplify permission assignment by grouping users with similar job functions.

Q5. An organisation needs to enforce that all IAM users must use MFA before performing any action in the AWS Console. How can this be implemented?

- A) Enable MFA on the root account only
- B) Attach an IAM policy to all users that denies all actions when MFA is not present
- C) Configure the Password Policy to require MFA
- D) Enable AWS Config to detect when users access the console without MFA

Answer: B

Explanation: An IAM policy with a Condition denying all actions when `aws:MultiFactorAuthPresent` is false enforces MFA for all console actions. Password Policy controls password complexity, not MFA. Config detects but does not prevent access.

Q6. What is the correct ARN format for an S3 bucket named `company-logs`?

- A) `arn:aws:s3:us-east-1:123456789012:company-logs`
- B) `arn:aws:s3:::company-logs`
- C) `arn:aws:s3:*:*:company-logs`
- D) `arn:s3:::company-logs`

Answer: B

Explanation: S3 bucket ARNs have no Region and no Account ID in the format — because bucket names are globally unique. The correct format is `arn:aws:s3:::bucket-name`.

Q7. A security audit finds that several IAM users have access keys that were created 18 months ago and have never been used. According to IAM best practices, what should be done?

- A) Rotate the access keys to new values
- B) Delete the unused access keys
- C) Move the users to a restricted group
- D) No action needed — unused keys cannot be exploited

Answer: B

Explanation: According to IAM best practices and the principle of least privilege, credentials that are not used should be removed entirely. An unused key that exists can still be stolen and exploited

— "never used" is not the same as "safe."

Q8. Which IAM policy type is DIRECTLY embedded within a specific user and cannot be reused elsewhere?

- A) AWS Managed Policy
- B) Customer Managed Policy
- C) Inline Policy
- D) Resource-Based Policy

Answer: C

Explanation: Inline policies are directly embedded in a single IAM entity (user, group, or role). They cannot be attached to other entities and are deleted when the entity is deleted. Managed policies (AWS or Customer) exist independently and can be attached to multiple entities.

Q9. A Lambda function needs to write logs to CloudWatch Logs and read items from a DynamoDB table. How should this be configured?

- A) Create an IAM User for the Lambda function and embed its access keys in the function code
- B) Create an IAM Role with CloudWatch Logs and DynamoDB permissions, and attach it as the Lambda execution role
- C) Give the Lambda function root account credentials
- D) Use the default Lambda role which has access to all AWS services

Answer: B

Explanation: Lambda functions use execution roles — IAM Roles that Lambda assumes when invoked. The role grants only the permissions needed (least privilege). There is no "default role with all access" — you specify the permissions.

Short Answer — Interview Style

Q10. What is the difference between an IAM Role and an IAM User?

Answer: An IAM User represents a specific person or application with permanent credentials (username/password and/or access keys). An IAM Role is an identity that can be temporarily assumed by an entity — a person, AWS service, or another account. Roles do not have permanent credentials; instead, they issue temporary credentials via STS that automatically expire. Roles are preferred for AWS services (EC2, Lambda) and cross-account access because they are more secure and do not require storing long-term credentials.

Q11. Explain the principle of least privilege and how you would implement it in AWS IAM.

Answer: Least privilege means granting only the permissions a user or service needs to perform its specific job — nothing more. In AWS IAM, you implement this by: (1) starting with no permissions and adding only what is required, (2) using Customer Managed Policies with specific service actions and resource ARNs rather than *, (3) using the IAM Access Analyser to identify

overly permissive policies, (4) reviewing the IAM Credential Report to remove unused permissions, and (5) using the Policy Simulator to verify effective permissions match the intent.

Q12. A user has an `Allow` for `s3:*` from their group policy AND a `Deny` for `s3:DeleteObject` from an SCP applied by their AWS Organization. Can the user delete S3 objects? Explain your reasoning.

Answer: No. An explicit `Deny` always overrides any `Allow` in AWS IAM. The SCP's explicit `Deny` for `s3:DeleteObject` wins over the group policy's `Allow` for `s3:*`. This is one of the reasons SCPs are used for organisational guardrails — they cannot be overridden by any identity-based policy in member accounts, ensuring organisation-wide enforcement of critical restrictions like preventing production data deletion.

Q13. What is a resource-based policy, and how does it differ from an identity-based policy? Give an example.

Answer: An identity-based policy is attached to an IAM entity (user, group, or role) and defines what that entity can do. A resource-based policy is attached directly to an AWS resource (like an S3 bucket) and defines who can access that resource and what they can do with it. The key difference is that resource-based policies can grant access to principals from other AWS accounts without requiring role assumption. Example: An S3 bucket policy can say "allow the IAM user Alice from account 123456789012 to download objects" — Alice in account A can access the bucket in account B just by the bucket policy, without any role assumption.

Q14. What does the IAM Credential Report contain, and when would you use it?

Answer: The IAM Credential Report is a CSV file listing all IAM users in your account with the status of their credentials: whether they have a console password and when it was last used, whether MFA is enabled, whether they have access keys and when those keys were last used and rotated, and whether keys are active or inactive. You use it during: (1) security audits — to find users with old or unused credentials, (2) compliance checks — to verify MFA is enabled for all users, (3) offboarding processes — to confirm a departed employee's access is fully revoked, and (4) automated security tooling — generating the report via API and parsing it for anomalies.

Summary — Chapters 1 & 2

Key Concepts to Remember for SAA-C03

Chapter 1 Takeaways

Concept	What to Remember
Cloud models	IaaS (EC2) = you manage OS; PaaS (RDS) = AWS manages platform; SaaS = AWS manages everything
Regions	Geographically separate; data doesn't auto-replicate between them
AZs	Fault isolation within a Region; always design for multi-AZ
Edge Locations	Used by CloudFront and Route 53; 400+ globally
Shared Responsibility	EC2 = you own OS; RDS = AWS owns engine; S3 = you own access control
Personal Health Dashboard	Account-specific events; subscribe to alerts
Well-Architected	6 pillars: OpEx, Security, Reliability, Performance, Cost, Sustainability

Chapter 2 Takeaways

Concept	What to Remember
IAM Users	Permanent credentials; one per person
IAM Groups	Permission management at scale; no login; no nesting
IAM Roles	Temporary credentials; best for AWS services; cross-account access
Policies	JSON documents; explicit Deny always wins over Allow
ARNs	Unique identifier for every AWS resource; used in policy Resource fields
MFA	Enable on root immediately; enforce on all privileged users
Least Privilege	Grant minimum permissions; audit regularly
Policy Simulator	Test permissions without making real API calls

What's Next

Chapter 3 — AWS CLI, SDK & Access Methods covers how to interact with AWS programmatically: installing the CLI, configuring profiles, understanding credential precedence, and using CloudShell. These skills directly apply to the labs in all subsequent chapters.

Guide prepared for AWS SAA-C03 Roadmap | Phase 1 — Cloud Foundations | Units U00 & U01